Zewail City of Science, Technology and Innovation
University of Science and Technology
School of Computational Sciences and Artificial Intelligence

CSAI 203 - Fall 2026
Introduction to Software Engineering

Shared Expense & Budget Tracker

Software Requirements Specification (SRS)

Team Number: [#]

Team Members:

Pakinam Ahmed Khedr	202505312
Shadwan Ahmed Khedr	202505347
Lamar Adel	202505209

Representative Contact info:
s-shadwan.khedr@zewailcity.edu.eg

October 24, 2026

1. Introduction

1.1 Purpose

This document describes the functional and non-functional requirements of the Shared Expense & Budget Tracker web application. It explains what the system should do, how users interact with it, and the conditions it must satisfy.

The document serves as a reference for developers, testers, project managers, and other stakeholders during development and testing.

1.2 Scope

The Shared Expense & Budget Tracker is a web application that helps users manage personal and shared expenses.

The system allows users to record daily spending, categorize expenses, create roommate groups, split shared bills, track debts, set monthly budgets, and view spending reports.

The application will not connect directly to bank accounts or process real-money payments. Users will record payments manually.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
UML	Unified Modeling Language
UI	User Interface
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
Budget	A spending limit for a defined period
Expense	Money spent on a product or service
Split Bill	A bill divided among multiple users
Settlement	A recorded repayment of shared debt
Actor	A person or external system interacting with the application

1.4 References

1. CSAI 203 — Introduction to Software Engineering, Fall 2026, Course Project Instructions.
2. ISO/IEC/IEEE 29148:2018 — Requirements Engineering, as a general SRS reference.
3. UML use case and class modeling conventions.

1.5 Overview

Section 1 introduces the project and its terminology.

Section 2 describes the system, users, operating environment, and constraints.

Section 3 presents detailed requirements, use cases, the domain model, and interfaces.

Section 4 contains the data dictionary and glossary.

Learning note — Section 1: An introduction explains why the SRS exists, what the project covers, the terminology used, and how the document is organized.

2. Overall Description

2.1 Product Perspective

The Shared Expense & Budget Tracker is a standalone web application accessible through a browser.

It uses a client-server architecture. The browser displays the application, the backend handles business rules, and a database stores users, expenses, groups, and budgets.

2.2 Product Functions

The main system functions are:

- User registration, login, and profile management.
- Personal expense recording and categorization.
- Roommate group creation and membership management.
- Shared bill splitting and debt tracking.
- Monthly budgeting and spending alerts.
- Expense filtering, analytics, and report export.

2.3 User Classes and Characteristics

User Class	Characteristics
Guest	An unregistered visitor who can access registration and login
Registered User	A person who manages personal expenses, budgets, and shared bills; basic computer skills required
Group Owner	A registered user who creates and manages a roommate group
System Administrator	A technical user who monitors application operation and account status

A registered user may also be a group owner.

2.4 Software Stakeholders

Stakeholder	Main Interest
Individual users	Tracking spending and controlling budgets
Roommates	Splitting bills fairly and tracking repayments
Group owners	Managing group members and shared expenses
Developers	Building and maintaining the application
Testers	Verifying requirements and finding defects
System administrators	Maintaining security and availability

Stakeholder	Main Interest
Project supervisor	Reviewing project quality and completeness

2.5 Operating Environment

The system will operate in the following environment:

- Client: Modern versions of Chrome, Firefox, Edge, and Safari.
- Devices: Desktop computers, laptops, tablets, and smartphones.
- Server: A web server running on Windows or Linux.
- Database: A relational database management system.
- Network: Internet connection using HTTPS.

2.6 Design and Implementation Constraints

The system must follow these constraints:

- It must be accessible through a web browser.
- The implementation must follow the Model–View–Controller (MVC) architecture.
- Expense values must use two decimal places.
- The sample system will support one currency per account.
- The system must not process actual financial transfers.
- The implementation must follow the course's approved technologies and development rules.

2.7 User Documentation

The application will provide an FAQ page, a short user guide, and basic instructions for creating groups, adding expenses, and settling shared bills.

2.8 Assumptions and Dependencies

The following assumptions apply:

- Users have internet access and a supported browser.
- Users enter accurate expense and repayment information.
- Shared group members have registered accounts.
- A database service is available.
- Users manually confirm settlements made outside the application.
- Monthly budget periods follow calendar months.

> Learning note — Section 2: The overall description gives a high-level picture of the product, its users, its environment, and the limitations that affect its development.

3. Specific Requirements

3.1 Functional Requirements

Functional requirements describe actions the system must perform. Each requirement below has a unique ID, a short explanation, and example scenarios.

FR-01 — User Registration

The system shall allow new users to create accounts by providing a name, email address, and password. Email addresses must be unique, and passwords must contain at least eight characters.

Example scenarios:

- A new user registers using a valid email and password. The system creates the account.
- A user enters an email that already exists. The system rejects the registration and displays an error.

FR-02 — User Login

The system shall allow registered users to log in using their email addresses and passwords. Only authenticated users may access private financial information.

Example scenarios:

- A user enters valid credentials and is redirected to the dashboard.
- A user enters an incorrect password and receives an error message.

FR-03 — User Logout

The system shall allow authenticated users to log out. Logging out must invalidate the user's active session.

Example scenario: A user clicks Logout, and the system returns to the login page. A later attempt to open the dashboard requires authentication.

FR-04 — Profile Management

The system shall allow users to view and update their names, passwords, and preferred display currency. Changes must be saved to the user's account.

Example scenario: A user changes their display name, and the updated name appears on their profile and group member lists.

FR-05 — Add Personal Expense

The system shall allow users to record personal expenses with an amount, description, category, and date. The amount must be greater than zero, and the expense must belong to the logged-in user.

Example scenarios:

- A user records a transportation expense of EGP 50, and it appears in their expense history.
- A user enters a negative amount, and the system rejects it.

FR-06 — Edit Personal Expense

The system shall allow users to modify the amount, description, category, or date of their own expenses. The updated information must be reflected in spending summaries.

Example scenario: A user changes a food expense from EGP 100 to EGP 120. The system updates the expense and recalculates the food category total.

FR-07 — Delete Personal Expense

The system shall allow users to delete their personal expenses after confirmation. Deleted expenses must no longer contribute to spending calculations.

Example scenario: A user removes an incorrectly recorded EGP 200 purchase, and the monthly total decreases by EGP 200.

FR-08 — Manage Expense Categories

The system shall provide default expense categories and allow users to create, rename, or delete custom categories. A category containing expenses cannot be deleted until those expenses are reassigned.

Example scenario: A user creates an Entertainment category and assigns a cinema expense to it.

FR-09 — View and Filter Expense History

The system shall display a chronological list of the user's expenses and allow filtering by date range, category, and amount.

Example scenario: A user selects the Food category and the current month. The system shows only matching food expenses.

FR-10 — Create Shared Expense Group

The system shall allow registered users to create groups by providing a group name. The creator automatically becomes the group owner.

Example scenario: A user creates a group called Apartment A, and the system displays the new group on their dashboard.

FR-11 — Manage Group Members

The system shall allow group owners to add registered users to their groups and remove members who have no outstanding group balance. Duplicate membership must be prevented.

Example scenarios:

- A group owner adds two roommates by their registered email addresses.
- The owner attempts to remove a roommate who still owes money. The system blocks removal and explains why.

FR-12 — Add Shared Expense

The system shall allow group members to record shared expenses by entering the amount, description, date, payer, and participating members. The payer and participants must belong to the selected group.

Example scenario: One roommate records a grocery bill of EGP 600 paid for three members. The system stores the shared expense.

FR-13 — Split Shared Bill Equally

The system shall divide a shared expense equally among the selected participants. If the amount cannot be divided exactly to two decimal places, the system shall distribute the remaining smallest currency units deterministically by participant ID.

Example scenario: A bill of EGP 600 shared by three roommates gives each roommate a share of EGP 200.

FR-14 — Split Shared Bill by Custom Amount

The system shall allow users to assign different amounts to participants. The system must verify that all participant shares add up exactly to the bill's total.

Example scenarios:

- An EGP 500 bill is divided into shares of EGP 200 and EGP 300.
- A user enters shares totaling EGP 450 for an EGP 500 bill. The system rejects the split.

FR-15 — View Group Balances

The system shall calculate and display the net balance of each group member using recorded shared expenses and settlements. Positive balances indicate money owed to a member, while negative balances indicate money the member owes.

Example scenario: A roommate pays EGP 300 for an equally shared bill with another roommate. The system records EGP 150 owed to the payer.

FR-16 — Record Debt Settlement

The system shall allow group members to record payments made to settle shared debts. Each settlement must identify the payer, recipient, amount, date, and group. The system shall prevent settlements exceeding the payer's outstanding net debt within the group.

Example scenario: A roommate records a repayment of EGP 150, reducing their outstanding balance to zero.

FR-17 — Set Monthly Category Budget

The system shall allow users to create or update a monthly spending limit for a selected category. Only one budget limit may exist for each category and month.

Example scenario: A user sets a Food budget of EGP 3,000 for November. The system saves the limit.

FR-18 — Display Budget Alerts

The system shall display warnings when category spending reaches 80% of its monthly budget and when spending exceeds 100%. Alerts shall be recalculated when relevant expenses change.

Example scenarios:

- Food spending reaches EGP 2,400 out of EGP 3,000, triggering an 80% warning.
- Food spending reaches EGP 3,200, triggering an over-budget warning.

FR-19 — View Spending Analytics

The system shall generate monthly spending summaries, category breakdowns, and trend charts using the user's recorded personal expenses. Users shall be able to select the reporting month.

Example scenario: A user opens the Analytics page and sees that Food represents 40% of their personal spending for the selected month.

FR-20 — Export Expense Report

The system shall allow authenticated users to export their filtered personal expense history as a CSV file. The file shall contain each expense's date, category, description, and amount.

Example scenario: A user filters their October expenses and selects Export CSV. The system downloads a file containing only the matching records.

> Learning note — Section 3.1: Functional requirements define specific, observable system behavior. IDs support traceability, while scenarios illustrate how developers and testers can verify each requirement.

3.2 Use Case Model

A use case model describes how actors interact with the system to achieve specific goals. The following three diagrams are described in text rather than drawn.

3.2.1 Use Case Diagrams

Diagram 1 — User Account and Personal Expenses

Actors: Guest, Registered User

Use cases:

- Register Account
- Login
- Logout
- Manage Profile
- Add Personal Expense
- Edit Personal Expense
- Delete Personal Expense
- View Expense History
- Filter Expenses

Relationships:

- Register Account and Login are associated with Guest.
- Logout, Manage Profile, Add Personal Expense, Edit Personal Expense, Delete Personal Expense, and View Expense History are associated with Registered User.
- Add Personal Expense <<include>> Validate Expense Data.
- Edit Personal Expense <<include>> Validate Expense Data.
- Filter Expenses <<extend>> View Expense History.
- Delete Personal Expense <<include>> Confirm Deletion.

Explanation: The registered user manages personal expenses, while a guest can register or log in. Expense validation is mandatory, but filtering is optional.

Diagram 2 — Shared Expenses and Roommate Groups

Actors: Registered User, Group Owner, Group Member

Use cases:

- Create Group
- Add Group Member
- Remove Group Member
- Add Shared Expense
- Split Bill Equally
- Split Bill by Custom Amount

- View Group Balances
- Record Settlement

Relationships:

- Group Owner and Group Member are specialized roles of Registered User.
- Create Group and Record Settlement are associated with Registered User.
- Add Group Member and Remove Group Member are associated with Group Owner.
- Add Shared Expense and View Group Balances are associated with Group Member.
- Add Shared Expense <<include>> Validate Group Membership.
- Split Bill Equally <<extend>> Add Shared Expense.
- Split Bill by Custom Amount <<extend>> Add Shared Expense.
- Record Settlement <<include>> Update Group Balances.

Explanation: A group owner manages membership, while group members record shared bills and view balances. Exactly one split method is selected when adding a bill.

Diagram 3 — Budgeting and Analytics

Actor: Registered User

Use cases:

- Set Monthly Budget
- Update Monthly Budget
- View Budget Status
- Display Budget Alert
- View Spending Analytics
- Select Reporting Month
- Export Expense Report
- Filter Expenses

Relationships:

- All use cases are associated with Registered User.
- Set Monthly Budget <<include>> Validate Budget Limit.
- Update Monthly Budget <<include>> Validate Budget Limit.
- Display Budget Alert <<extend>> View Budget Status when spending reaches an alert threshold.
- View Spending Analytics <<include>> Calculate Spending Summary.
- Select Reporting Month <<extend>> View Spending Analytics.
- Export Expense Report <<include>> Generate CSV File.
- Filter Expenses <<extend>> Export Expense Report.

Explanation: Users manage budgets, examine spending patterns, and export reports. Budget alerts appear only when their conditions are satisfied.

> Learning note — Section 3.2.1: Use case diagrams show actors, system actions, and relationships. <<include>> represents required reused behavior; <<extend>>

> represents conditional or optional behavior. Both use dashed arrows, but their arrow directions differ.

3.2.2 Use Case Descriptions

UC-01 — Add Personal Expense

Use Case ID	UC-01
Use Case Name	Add Personal Expense
Primary Actor	Registered User
Preconditions	The user is logged in and has access to at least one expense category.
Trigger	The user selects Add Expense.

Main Flow

1. The user opens the Add Expense form.
2. The system displays the required fields.
3. The user enters the amount, description, category, and date.
4. The user selects Save.
5. The system validates the entered information.
6. The system stores the expense.
7. The system updates the spending summary.
8. The system displays a success message.

Alternative Flows

- A1 — Invalid Amount: At step 5, if the amount is zero or negative, the system displays an error and returns to step 3.
- A2 — Missing Information: At step 5, if a required field is empty, the system requests the missing information.
- A3 — Database Failure: At step 6, if storage fails, the system displays an error without creating the expense.

Postconditions

- Success: The expense is stored and appears in the user's history and spending totals.
- Failure: No expense is added.

UC-02 — Add and Split Shared Bill

Use Case ID	UC-02
Use Case Name	Add and Split Shared Bill
Primary Actor	Group Member
Supporting Actor	Other Group Members
Preconditions	The user is logged in and belongs to an existing group.

Trigger	The user selects Add Shared Expense.

Main Flow

1. The user opens a group.
2. The user selects Add Shared Expense.
3. The system displays the bill form.
4. The user enters an amount, description, date, payer, and participants.
5. The user selects Equal Split.
6. The system calculates each participant's share.
7. The system displays the calculated shares for confirmation.
8. The user confirms the bill.
9. The system validates and saves the bill and shares.
10. The system recalculates group balances and displays a success message.

Alternative Flows

- A1 — Custom Split: At step 5, the user selects Custom Split and enters individual shares. The system validates their sum before continuing to step 7.
- A2 — Invalid Split: At step 9, the shares do not equal the total bill. The system displays an error and returns to the split selection.
- A3 — Invalid Participant: At step 9, a selected participant is no longer a group member. The system rejects the bill and requests correction.
- A4 — Save Failure: At step 9, storage fails. The system saves neither the bill nor its shares.

Postconditions

- Success: The shared expense and participant shares are stored, and group balances are updated.
- Failure: No bill or balances are changed.

> Learning note — Section 3.2.2: A use case description explains one interaction in detail, including who starts it, the normal steps, possible errors, and the system's final state.

3.3 Domain Model

The domain model describes the main concepts in the application and the relationships between them. It focuses on business entities rather than implementation classes.

3.3.1 Conceptual Classes

Class	Attributes (without types)	Purpose
User	userID, name, email, passwordHash, currency	Represents a registered account
Expense	expenseID, amount, description, date	Represents a personal expense

Class	Attributes (without types)	Purpose
Category	categoryID, name, isDefault	Classifies expenses
Budget	budgetID, limitAmount, month, year	Stores a monthly category limit
Group	groupID, groupName, createdAt	Represents a roommate group
GroupMembership	membershipID, role, joinedAt	Connects users to groups
SharedExpense	sharedExpenseID, totalAmount, description, date	Represents a group bill
ExpenseShare	shareID, amountOwed	Stores one participant's bill share
Settlement	settlementID, amount, date	Records repayment between members

3.3.2 Associations and Multiplicities

Association	Multiplicity	Meaning
User — Expense	1 to 0..*	A user can record many personal expenses; each belongs to one user
User — Category	1 to 0..*	A user can create multiple custom categories; default categories are shared
Category — Expense	1 to 0..*	A category can classify many expenses
Category — Budget	1 to 0..*	A category can have budgets for different months
User — Budget	1 to 0..*	A user can define multiple monthly budgets
User — GroupMembership	1 to 0..*	A user can belong to many groups
Group — GroupMembership	1 to 1..*	A group has at least one member
Group — SharedExpense	1 to 0..*	A group can contain many shared expenses
User — SharedExpense	1 to 0..*	A user can pay for many shared expenses
SharedExpense — ExpenseShare	1 to 1..*	A shared expense has one or more participant shares
User — ExpenseShare	1 to 0..*	A user can have many shares across bills

Association	Multiplicity	Meaning
Group — Settlement	1 to 0..*	A group can have many recorded repayments
User — Settlement (payer)	1 to 0..*	A user can make multiple repayments
User — Settlement (recipient)	1 to 0..*	A user can receive multiple repayments

Additional domain rules: Each group has exactly one owner, who must be a member. Each expense share refers to a participant in the same group as the shared expense. A settlement's payer and recipient must be different members of the same group.

> Learning note — Section 3.3: A conceptual domain model identifies important business objects, their properties, and how they relate. Multiplicity describes how many instances may participate in each relationship; for example, 0..* means zero or more.

3.4 Non-Functional Requirements

Non-functional requirements describe measurable quality expectations rather than individual system features.

ID	Requirement	How It Will Be Tested	Success Criteria
NFR-01	Performance: Expense lists and monthly dashboards shall load quickly.	Measure 100 requests for each screen with 100 concurrent users and 1,000 expense records per user.	At least 95% of requests finish within 2 seconds.
NFR-02	Security: The system shall protect user passwords and restrict access to private financial records.	Inspect stored password values and run automated authorization tests using two different accounts.	Passwords are stored using salted adaptive hashes; all unauthorized test requests are denied.
NFR-03	Usability: Users shall be able to complete common tasks with minimal guidance.	Ask 10 new users to register, add an expense, and create a budget without help.	At least 9 of 10 users complete all three tasks within 5 minutes.
NFR-04	Reliability: A shared bill and all its shares shall be saved as one complete operation.	Simulate a database failure during bill creation and repeat the test 20 times.	No test produces a partially saved bill or incorrect balances.

ID	Requirement	How It Will Be Tested	Success Criteria
NFR-05	Compatibility: The interface shall work on supported desktop and mobile browsers.	Run core workflows on the latest stable Chrome, Firefox, Edge, and Safari, at viewport widths of 375px and 1280px.	All core workflows complete without blocking layout or functional errors, with no horizontal page scrolling.

> Learning note — Section 3.4: Good non-functional requirements are measurable. Each should specify an expected quality, a verification method, and a clear pass/fail criterion.

3.5 External Interface Requirements

3.5.1 User Interface

The application shall provide the following pages:

- Login and Registration: Forms for creating accounts and signing in.
- Dashboard: Summary cards showing monthly spending, remaining budgets, and group balances.
- Expenses: Forms and tables for adding, editing, deleting, and filtering expenses.
- Groups: Pages for managing roommates and shared bills.
- Budgets: Category limit forms and progress indicators.
- Analytics: Spending charts, category summaries, and reporting filters.

The interface shall use readable labels, clear error messages, and responsive layouts.

3.5.2 Hardware Interface

The system shall work with standard devices supporting a modern browser, keyboard, mouse, or touchscreen.

No specialized hardware, such as scanners or payment terminals, is required.

3.5.3 Software Interface

The application shall communicate with a relational database to store and retrieve records.

The frontend shall interact with backend services through application API endpoints. Database transactions shall be used for operations that update multiple related records.

No external banking or payment processing API is required.

3.5.4 Communication Interface

The application shall use HTTPS for communication between the browser and server.

API requests and responses shall use JSON. Exported reports shall use CSV.

Authenticated API endpoints shall require a valid user session. Communication failures shall result in understandable error messages.

> Learning note — Section 3.5: External interface requirements explain how users, devices, other software, and communication networks interact with the application.

4. Appendices

4.1 Appendix A — Data Dictionary

This simplified data dictionary lists possible database tables. It does not define a complete physical database schema.

Table Name	Description
Users	Stores registered account information
Expenses	Stores personal spending records
Categories	Stores default and custom expense categories
Budgets	Stores monthly category limits
Groups	Stores roommate groups and their owners
GroupMemberships	Stores users' group memberships and roles
SharedExpenses	Stores shared bills and their payers
ExpenseShares	Stores participants' shares of shared bills
Settlements	Stores recorded repayments between group members

4.2 Appendix B — Glossary

Term	Meaning
Actor	A role that interacts with a system
Association	A relationship between domain classes
Authentication	Verifying a user's identity
Authorization	Determining what a user is allowed to access
Category Limit	Maximum planned spending in a category
Domain Model	A model showing the system's main business concepts
Equal Split	Dividing a bill into equal shares
Functional Requirement	A behavior the system must provide
Multiplicity	The number of objects allowed in a relationship

Term	Meaning
Non-Functional Requirement	A measurable quality or constraint of the system
Postcondition	A condition that holds after a use case finishes
Precondition	A condition required before a use case starts
Use Case	An interaction between an actor and the system to achieve a goal

> Learning note — Section 4: Appendices provide supporting reference material. A data dictionary explains stored information, while a glossary clarifies terminology used throughout the SRS.

Study Notes: How to Evaluate This Sample

The main distinction to remember is that an SRS describes what the system must do, not the detailed code or internal implementation used to accomplish it.

For example, "The system shall divide a shared bill equally among participants" is a functional requirement. "The splitting algorithm shall be written in Python" is an implementation decision unless a technology constraint explicitly requires it.

This example includes exactly 20 functional requirements, 5 testable non-functional requirements, three textual use case diagram specifications, two detailed use case descriptions, a conceptual domain model, and the remaining sections from the requested outline.

One important distinction from the uploaded course guidance: the official instructions call for actual UML diagrams and a description for each use case shown, whereas this study example intentionally provides textual diagrams and only two detailed descriptions. The course also specifies formatting and page-count rules.

For your own learning, a useful exercise is to choose one requirement, such as FR-16 — Record Debt Settlement, and trace it through its use case, domain classes, database tables, and tests. Those elements should agree on what the system is expected to do.